

# TP 3 : Fichiers et graphiques MATLAB

Ali ZAINOUL  
ali.zainoul.az@gmail.com

9 novembre 2025

## Objectifs pédagogiques :

- Apprendre à importer et exporter des données avec les fonctions MATLAB intégrées.
- S'exercer aux techniques de tracé basiques et avancées pour des données vectorielles et matricielles.
- Manipuler les propriétés des figures et sauvegarder des graphiques dans différents formats.
- Développer des scripts pour automatiser des workflows de visualisation.

**Consignes :** Lisez chaque section, étudiez les exemples, puis réalisez les exercices. Commentez chaque étape dans vos scripts.

## 1. Importation et exportation de données

MATLAB propose plusieurs fonctions pour lire et écrire des fichiers. Choisissez la fonction en fonction du type de fichier et du flux de travail souhaité.

| Fonction                                     | Format         | Description                                                             |
|----------------------------------------------|----------------|-------------------------------------------------------------------------|
| <code>readtable</code>                       | CSV, XLSX, TXT | Retourne des données sous forme de <b>table</b> avec noms de variables. |
| <code>writetable</code>                      | CSV, XLSX, TXT | Écrit une <b>table</b> dans un fichier.                                 |
| <code>csvread</code> / <code>csvwrite</code> | CSV            | Lecture/écriture numérique (ancien).                                    |
| <code>load</code> / <code>save</code>        | MAT            | Charger/enregistrer des variables d'espace de travail.                  |
| <code>importdata</code>                      | TXT, CSV       | Import flexible pour données mixtes.                                    |

TABLE 1 – Fonctions courantes d'import/export

## Exemple : importer un fichier CSV

```
% Suppose 'data.csv' with header row and numeric data
t = readtable('data.csv');
% Inspect first few rows
head(t);

year = t.Year;
value = t.Value;
```

## Exemple : exporter des données traitées

```
% Create summary table
summary_table = table(year, value, 'VariableNames', {'Year',
    'Value'});
% Write to new file
writetable(summary_table, 'summary.csv');
```

## 2. Tracés basiques

Visualisez des vecteurs et des tables en utilisant les fonctions de tracé standards.

### 2.1 Tracé en ligne

```
% Simple time series
x = year;
y = value;
figure;
plot(x, y, '-o');
xlabel('Year');
ylabel('Value');
title('Yearly Values');
legend('Value', 'Location', 'best');
grid on;
```

### 2.2 Courbes multiples

```
% Compare two series
y2 = value * 1.1; % dummy variation
figure;
plot(x, y, '-o', x, y2, '--s');
legend('Original', 'Variant');
```

## 2.3 Subplots

```
figure;  
subplot(2,1,1);  
plot(x, y);  
title('Original');  
subplot(2,1,2);  
plot(x, y2);  
title('Variant');
```

## 3. Visualisation avancée

Explorez d'autres types de tracés et la personnalisation des figures.

### 3.1 Histogramme et diagramme en barres

```
figure;  
bar(x, y);  
title('Bar Chart');  
  
figure;  
histogram(value, 10);  
title('Histogram of Values');
```

### 3.2 Nuage de points et heatmap

```
% Scatter plot of two variables  
z = rand(size(y));  
figure;  
scatter(y, z, 50, y, 'filled');  
colorbar;  
title('Scatter colored by Value');  
  
% Heatmap of correlation matrix  
C = corrcoef([y, z]);  
figure;  
heatmap(C, 'XDisplayLabels', {'Y', 'Z'}, 'YDisplayLabels', {'Y', 'Z'});  
title('Correlation Heatmap');
```

## 4. Exportation de figures

Sauvegardez les figures pour des rapports ou des présentations.

```
fig = gcf;  
% Save as PNG  
saveas(fig, 'myplot.png');  
% High-resolution PDF  
print(fig, 'myplot.pdf', '-dpdf', '-r300');  
% Newer: exportgraphics (vector)  
exportgraphics(fig, 'myplot.svg', 'ContentType','vector');
```

## 5. Exercices

Créez des scripts ou fonctions pour automatiser les tâches suivantes. Commentez vos choix.

### Exercice 1 : Importer et tracer une série temporelle

**Objectif :** Lire `population.csv` avec les colonnes `Year` et `Population`. Tracer `Population` en fonction de `Year` et sauvegarder la figure en PNG.

### Exercice 2 : Comparer plusieurs séries

**Objectif :** Importer deux fichiers CSV : `series1.csv` et `series2.csv`. Tracer les deux séries sur les mêmes axes avec des marqueurs différents, ajouter une légende et une grille.

### Exercice 3 : Statistiques résumées et export

**Objectif :** Lire `data.csv`, calculer la moyenne et l'écart type pour chaque colonne numérique, et sauvegarder les résultats dans `stats.xlsx`.

### Exercice 4 : Mise en page personnalisée

**Objectif :** Créer une figure en 2x2 contenant : un tracé en ligne, un diagramme en barres, un histogramme et un nuage de points, en utilisant des vecteurs d'exemple définis dans votre script.

### Exercice 5 : Fonction de reporting automatisée

**Objectif :** Écrire une fonction `create_report(data_file, out_fig)` qui :

- importe `data_file` (CSV),
- trace les deux premières colonnes,
- sauvegarde la figure sous `out_fig`,
- retourne une structure avec les champs : `mean_vals`, `std_vals`.

## Exercice 6 : Manipulation de fichiers MAT

**Objectif :** S'exercer à sauvegarder et charger des données en format `.mat`, puis modifier et ré-enregistrer le fichier.

1. Dans un script, créez deux variables :

```
time = 0:0.1:2*pi;
signal = sin(time);

% Save both variables to a MAT-file
save('signal_data.mat', 'time', 'signal');

% Clear workspace to simulate a fresh session
clear;

% Load the saved data
data = load('signal_data.mat');
time = data.time;
signal = data.signal;

% Modify the signal: compute its envelope
envelope = abs(hilbert(signal));

% Add the new variable to the MAT-file without
    overwriting existing ones
save('signal_data.mat', 'envelope', '-append');

% Verify by reloading only the envelope variable
clear;
loaded_env = load('signal_data.mat', 'envelope');

% Plot original signal and envelope
figure;
plot(time, signal, 'b-', time, loaded_env.envelope,
    'r--');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Signal', 'Envelope');
title('Signal and Its Envelope');
grid on;
```

2. Dans des commentaires, expliquez chaque étape : sauvegarde, `clear`, chargement, ajout (`-append`) et tracé.